PROJECT REPORT OF

Meridian — AI Multi-Agent Market Intelligence Platform

Submitted in partial fulfillment of the requirements for the award of

DEGREE OF BACHELOR OF COMPUTER APPLICATION

SUBMITTED BY:

Name: Mridul Sharma

Class: BCA - 5

University Roll No.: 2430087

SUBMITTED TO:

DEPARTMENT OF COMPUTER APPLICATIONS

GLOBAL GROUP OF INSTITUTES, AMRITSAR

AUGUST – DECEMBER, 2026

DECLARATION

I hereby declare that the project work entitled “Meridian — AI Multi-Agent Market Intelligence Platform” is an authentic record of my own work carried out at Global Group of Institutes, Amritsar (affiliated to I.K. Gujral Punjab Technical University, Jalandhar) as requirement for the six months project for the award of the degree of BCA, under the guidance of Mr. Amit Puri, during August to December 2026.

Mridul Sharma
Class: BCA - 5
(University Roll No.: 2430087)

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Signature of the SUPERVISOR(S)

Mr. Amit Puri (HOD, CA Dept.)

The Viva-Voce Examination of the student has been held on _____ and accepted.

Examined by:

Head of Department

ACKNOWLEDGMENT

Through this acknowledgment, I express my sincere gratitude towards all those people who have helped me in this project, which has been a rich learning experience.

This space wouldn't be enough to extend my warm gratitude towards my project supervisor for efforts in coordinating with my work and guiding me in the right direction.

First and foremost, I would like to thank our Campus Dr. M. S. SAINI, Director, Global Group of Institutes, Amritsar for permitting me to carry out the project work.

I escalate a heartfelt gratitude to our respected Head of the Department Mr. Amit Puri (HOD, CA Dept.) and Ms. Komalpreet Kaur for giving me the essential technical insight and creative freedom to build this project.

Finally, I also thank all faculty members in the Department of Computer Applications, Global Group of Institutes, Amritsar, who helped me directly or indirectly in accomplishing this work.

Mridul Sharma
Class: BCA - 5
(Roll No.: 2430087)

(Signature)

List of Contents

S.NO	TITLE	PAGE.NO
1.	Introduction to Project	5 - 6
2.	Objective and Scope of Project	7 - 8
3.	System Analysis, Architecture & UML Diagrams	9 - 12
4.	Introduction to HTML5, CSS3, JavaScript, Python & VS Code	13 - 14
5.	Project Code	15 - 21
6.	Screenshots of Project	22 - 24
7.	Software Testing & Error Handling	25 - 26
8.	Conclusion & Future Scope	27
9.	References	28

CHAPTER 1: INTRODUCTION TO PROJECT

1.1 Overview of Meridian

Market intelligence — the disciplined collection and analysis of information about markets, competitors, customers, and macroeconomic forces — has become a decisive input to business strategy. Consulting-grade research reports that once required teams of analysts and weeks of effort are now expected within hours, and decision makers increasingly ask follow-up questions of the data itself. At the same time, general-purpose Artificial Intelligence (AI) chatbots frequently invent figures, fabricate citations, and present guesses as verified facts, which makes them unsafe for board-level decisions.

Meridian — AI Multi-Agent Market Intelligence Platform is a full-featured, multi-agent web application (Software-as-a-Service architecture) engineered using Python (FastAPI), HTML5, CSS3, Vanilla JavaScript (ES6+), and SQLite / PostgreSQL that solves this problem end-to-end. A user simply enters a research query such as “India's economy growth compared to China”, selects a Region, Period, and Language, and clicks Generate Intelligence. Within moments, seven autonomous AI agents — Researcher, Analyst, Writer, Forecast Agent, Fact-Checker, Chat Agent, and Translator — collaborate to research live sources, run SWOT / PESTLE / Porter's Five Forces analyses, forecast twelve months of scenarios, fact-check every claim, and compose a board-ready report with real citations, interactive charts, and one-click PDF, PowerPoint, and Excel exports.

A defining architectural innovation of Meridian is its Verified-Answer Policy combined with a Bring-Your-Own-Key (BYOK) multi-provider engine. Users connect free-tier API keys from sixteen supported providers — OpenRouter (free models), Groq, Google Gemini, Cerebras, Mistral, Cohere, OpenAI, Anthropic, and others — directly inside the Settings interface. Keys are encrypted at rest with Fernet (AES-128-CBC + HMAC) cryptography, and no answer is ever manufactured from templates: if no API key or no live evidence is available, Meridian refuses with a clear, structured error instead of publishing fabricated research.

1.2 Problem Statement & Limitations of Existing Solutions

Before designing Meridian, existing approaches to market research were analyzed carefully, revealing four persistent limitations:

- **Slow, Expensive Manual Research:** Agency and consulting reports cost thousands of rupees/dollars and take days to weeks, putting them out of reach of students, founders, and small teams.

• **Hallucinating AI Chatbots:** General-purpose chatbots invent market sizes, company shares, and citations bearing real publisher names, and present the guesses with total confidence — dangerous for academic and business decisions.

• **Expensive Subscription SaaS:** Commercial research platforms typically require recurring monthly subscriptions ($20–$100+/month), mandatory enterprise onboarding, and per-seat licensing.

• **Privacy & Credential Risks:** Storing API keys and research data on third-party servers in plaintext exposes users to breaches; many tools also resell usage analytics.

1.3 Proposed System & Key Architectural Innovations

Meridian overcomes every limitation above through a verified, multi-agent SaaS architecture:

• **Seven Specialised AI Agents:** Researcher, Analyst, Writer, Forecast Agent, Fact-Checker, Chat Agent, and Translator execute as an orchestrated pipeline with live mission-control telemetry instead of one opaque chatbot call.

• **Verified-Answer Policy (Fail-Closed):** Without a configured provider API or live evidence, Meridian returns HTTP 409 and generates nothing — it never substitutes a template answer, and an explicit, loudly-labelled sample mode is the only offline exception.

• **100% Free Bring-Your-Own-Key Multi-AI Engine:** Sixteen providers across seven independent agent slots; OpenRouter models are auto-selected live from the catalogue, restricted to genuine :free variants, with all-free fallback chains — never a paid model.

• **Free-Tier Failure Survival:** Typed handling of empty, refused, reasoning-only, and truncated completions; truncated-JSON repair; reasoning-disabled retries; HTTP 429 cooldown windows honouring Retry-After; and failover to another configured free provider.

• **Encrypted Key Vault & Exports:** Fernet-encrypted credentials, masked UI, and one-click PDF / PowerPoint / Excel report generation.

CHAPTER 2: OBJECTIVE AND SCOPE OF PROJECT

2.1 Primary Objectives of Meridian

• To architect and implement a clean, responsive, multi-view SaaS web application using core academic web technologies: HTML5, CSS3, Vanilla JavaScript, and Python (FastAPI).

• To transform one natural-language query into a complete market-intelligence report containing an executive summary, market trajectory and competitive-share charts, SWOT, PESTLE, Porter's Five Forces, key metrics, and strategic priorities.

• To guarantee factual integrity: every figure must trace to a live retrieved source or be labelled 'not available'; reliability is scored claim-by-claim by an independent Fact-Checker agent.

• To build a 12-month base / optimistic / pessimistic forecast with confidence bands, and a grounded Chat Agent that answers follow-up questions only from the report's verified context.

• To support report translation into 14 languages and side-by-side comparison of two companies or two saved reports.

• To make the entire platform operable at zero API cost through free-tier providers and genuine :free OpenRouter model auto-selection.

2.2 Scope of the Project

Meridian's scope spans the complete research workflow — query, evidence collection, analysis, writing, forecasting, verification, export, follow-up chat, comparison, and translation — for any industry, company, region (Global, India, USA, Europe, Asia-Pacific and more), and period (1Y, 3Y, 5Y). Out of scope are investment advice, trading execution, and paywalled data licensing; where evidence cannot be retrieved, the product says so explicitly instead of estimating silently.

2.3 Target Beneficiaries

• **University Students & Researchers:** Producing cited market studies for projects, dissertations, and startup competitions without paid databases.

• **Founders & Small Businesses:** Validating market size, competitors, and entry strategy before spending on agencies.

• **Analysts & Consultants:** Drafting a first-pass, fully-cited report in minutes and refining it through follow-up chat and exports.

• **Corporate Teams:** Comparing competitors and tracking sector trends with an auditable trail of sources and reliability scores.

CHAPTER 3: SYSTEM ANALYSIS, ARCHITECTURE & UML DIAGRAMS

3.1 Project Category

Web-Based Artificial Intelligence SaaS Application (Layered Client–Server Architecture powered by HTML5, CSS3, ES6 JavaScript, Python 3 FastAPI, a seven-agent orchestration engine, and RESTful free-tier Large Language Model APIs).

3.2 Feasibility Analysis

• **Technical Feasibility:** Uses lightweight, universally supported web standards and Python's HTTP ecosystem; no dedicated GPUs or database clusters are required, and the server runs comfortably on a free PaaS instance.

• **Economic Feasibility:** Zero recurring API cost on free tiers; SQLite storage by default with PostgreSQL as a drop-in production upgrade.

• **Operational Feasibility:** A single query box with Region / Period / Language selectors; live progress telemetry makes long-running generation transparent, and every failure state explains the next step.

• **Schedule Feasibility:** The modular agent design allowed incremental development and testing — router, vault, agents, UI, and exports were verified independently before integration.

3.3 System Architecture

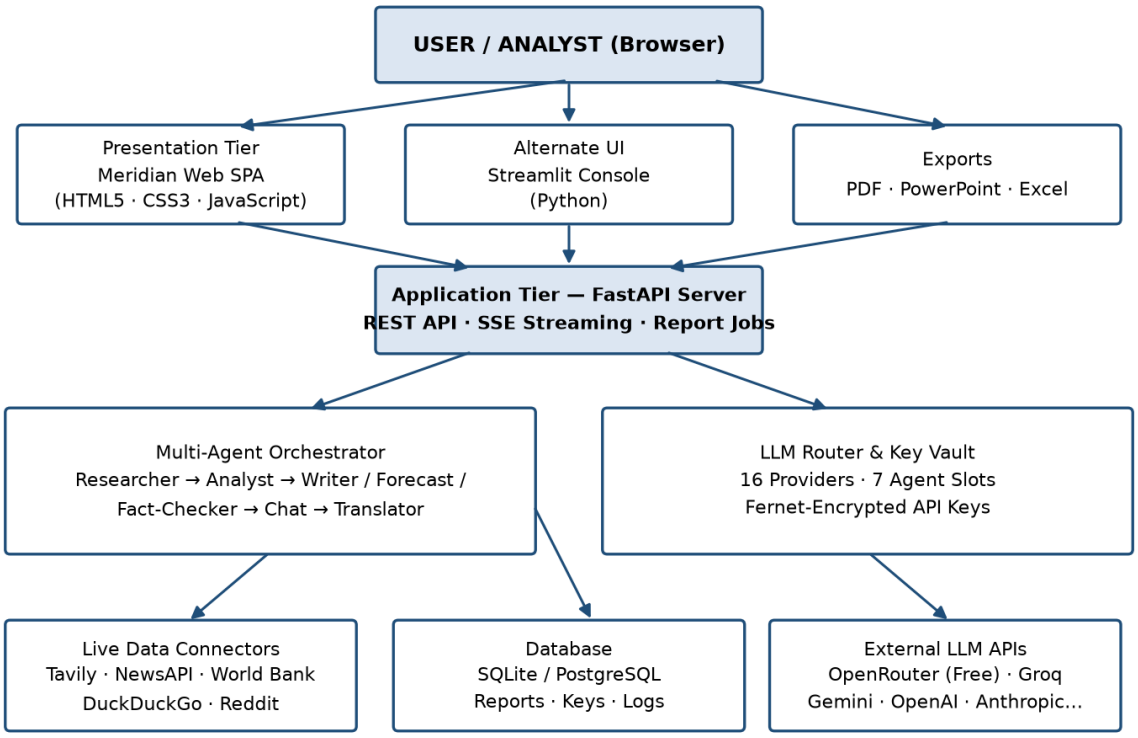


Figure 3.1: Meridian layered architecture — presentation, FastAPI application tier, multi-agent orchestrator, and external data planes

3.4 UML Use Case & Activity Workflow Diagrams

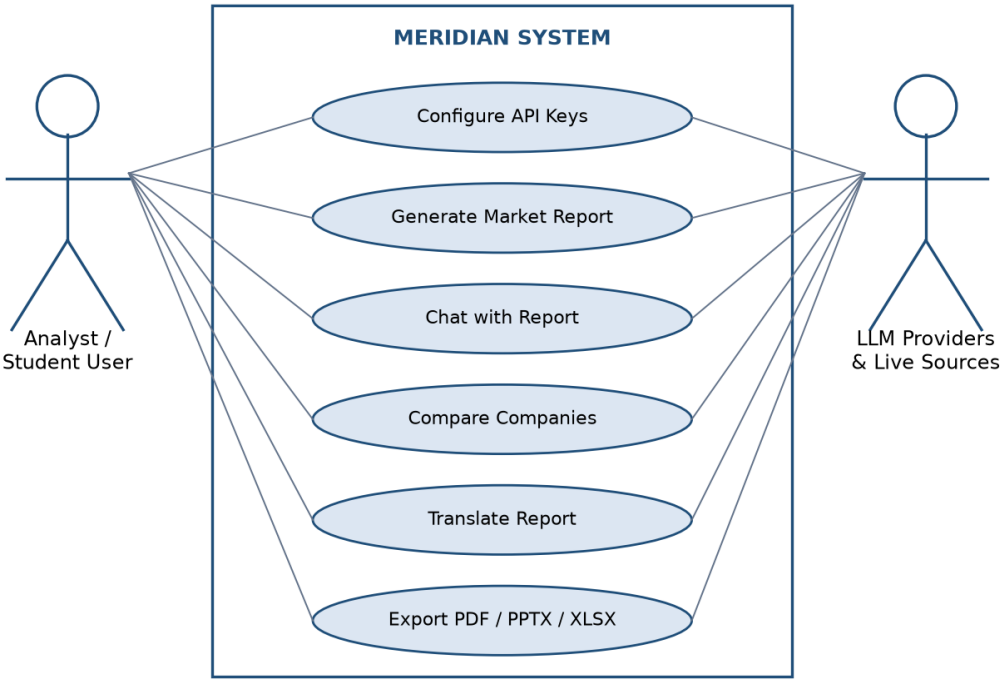


Figure 3.2: UML Use Case Diagram illustrating user, LLM-provider, and live-source interactions

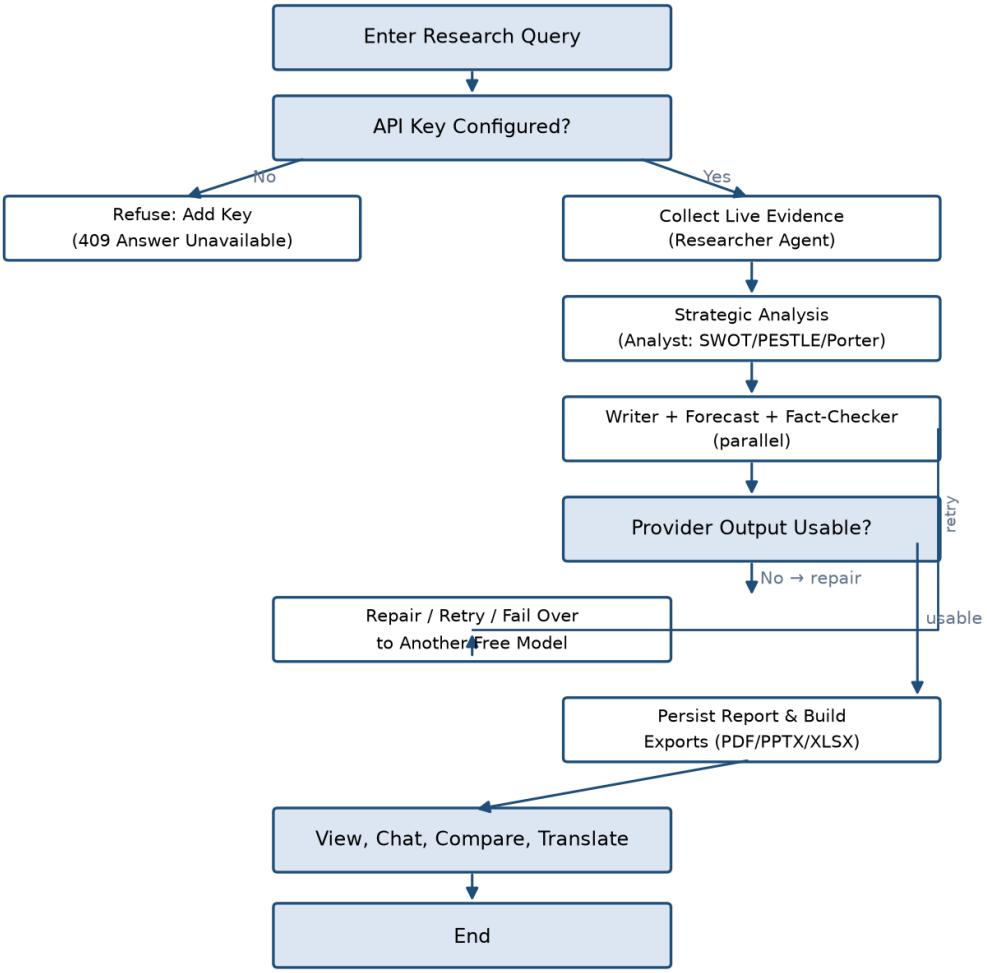


Figure 3.3: UML Activity Diagram depicting the verified report pipeline with repair, retry, and failover paths

3.5 Backend REST API Specification

Endpoint	Method	Purpose & JSON Response
/api/bootstrap	GET	App metadata, agent roster, and chat suggestions for first paint.
/api/reports	POST	Validates the query and starts an asynchronous seven-agent report job.
/api/reports/{id}/events	GET (SSE)	Streams agent_start / agent_retry / agent_done / agent_failed / job_complete events
/api/reports/{id}/status	GET	Polling fallback returning per-agent status and pipeline progress.
/api/reports/{id}/chat	POST	Grounded follow-up answer from the report's verified context only.
/api/reports/{id}/translate	POST	Full-report translation into any of the 14 supported languages.
/api/reports/{id}/export/{fmt}	GET	Downloads the generated PDF, PPTX, or XLSX export.
/api/compare	POST	Builds a side-by-side comparison of two companies.
/api/settings/keys	POST	Validates and Fernet-encrypts an API key into an agent slot.
/api/settings/keys/test	POST	Probes the key in the form; saves it only when the probe succeeds.

Endpoint	Method	Purpose & JSON Response
/api/settings/openrouter/models	GET	Live ranked catalogue of genuine :free OpenRouter models.
/api/stats	GET	Honest counters: completed reports, stored sources, agent executions.

CHAPTER 4: INTRODUCTION TO HTML5, CSS3, JAVASCRIPT, PYTHON & VS CODE

4.1 INTRODUCTION TO HTML5

HTML (HyperText Markup Language) is the standard markup language used to structure content on the World Wide Web. Originally proposed by Tim Berners-Lee in 1991 and standardized as HTML5 by the W3C and WHATWG, HTML5 introduces semantic structural tags, native form validation attributes, multimedia support, and rich accessibility hooks.

In Meridian, HTML5 provides the semantic foundation of the single-page application: a persistent sidebar and mission-control layout, the query composer, settings slot cards, history tables, and report sections, all served as one shell whose views are rendered by JavaScript.

4.2 INTRODUCTION TO CSS3

CSS3 (Cascading Style Sheets Level 3) separates presentation from structure. Meridian's stylesheet implements a strict three-colour design system — Obsidian #0B0906, Champagne #C6A15B, Pearl #F3ECDD — with CSS variables, grid layouts (g2/g3/g4), sticky panels, skeleton states, and breakpoint rules that guarantee the interface fits phones, tablets, laptops, and ultra-wide monitors without overlap or sideways scrolling.

4.3 INTRODUCTION TO JAVASCRIPT

JavaScript (ES6+) is the client-side programming language of the Web. In Meridian it implements hash routing across six views, consumes the Server-Sent Events stream to animate agent telemetry, draws line, donut, bar, and heatmap charts, manages the encrypted-key settings forms (including test-before-save), renders Markdown reports safely, and drives exports, comparison, chat, and translation workflows.

4.4 INTRODUCTION TO PYTHON

Python is a high-level, interpreted, general-purpose programming language prized for readability and its vast library ecosystem. Meridian's entire backend is Python 3.11: the FastAPI server (REST + SSE), the agent orchestration engine, the sixteen-provider LLM router, the free-model catalogue ranker, the Fernet key vault, live-data connectors, and the ReportLab / python-pptx / openpyxl export generators. A Streamlit interface provides an alternate Python-native console over the same services.

4.5 VISUAL STUDIO CODE

Visual Studio Code (VS Code) is a free, cross-platform source-code editor from Microsoft with an integrated terminal, Git client, debugger, and extension marketplace. The complete Meridian

codebase — Python services, the web SPA, tests, and deployment configuration (Render blueprint, Docker) — was authored, tested (123 automated tests), and version-controlled from VS Code.

CHAPTER 5: PROJECT CODE

This chapter presents the core source code of Meridian (excerpts). The complete, tested codebase is maintained in the project repository and deployed from the main branch.

5.1 LLM Router — typed completion extraction (app/api/llm_router.py)

```
# ---- app/api/llm_router.py (excerpt) ----
def _extract_completion(p: Provider, data: Dict[str, Any]) -> tuple[str, Dict[str, Any]]:
    """Return completion text plus finish/usage diagnostics.

    Providers increasingly return HTTP 200 for reasoning-only, refused or
    token-truncated responses. Treating all of those as a generic empty string
    made a single free model look like a dead API key, so the distinction is
    preserved here and in :func:`call_llm`'s return value.
    """
    if not isinstance(data, dict):
        raise LLMError(f"Malformed response from {p.label}: expected an object")
    usage = data.get("usage") if isinstance(data.get("usage"), dict) else {}
    meta: Dict[str, Any] = {
        "finish_reason": "",
        "reasoning_tokens": _usage_int(usage, "reasoning_tokens"),
        "completion_tokens": _usage_int(
            usage, "completion_tokens", "output_tokens"),
        "truncated": False,
    }
    try:
        if p.api_style == "anthropic":
            text = _content_text(data.get("content", []))
            meta["finish_reason"] = str(data.get("stop_reason") or "")
            meta["truncated"] = meta["finish_reason"] == "max_tokens"
            if text:
                return text, meta
            raise _completion_error(
                p, finish_reason="length" if meta["truncated"] else meta["finish_reason"],
                reasoning_tokens=meta["reasoning_tokens"],
                completion_tokens=meta["completion_tokens"])
        if p.api_style == "cohere":
            msg = data.get("message", {}) if isinstance(data.get("message"), dict) else {}
            text = _content_text(msg.get("content"))
            meta["finish_reason"] = str(data.get("finish_reason") or "")
            meta["truncated"] = meta["finish_reason"].upper() == "MAX_TOKENS"
            if text:
                return text, meta
            raise _completion_error(
                p, finish_reason="length" if meta["truncated"] else meta["finish_reason"],
                reasoning=_reasoning_text(msg),
                reasoning_tokens=meta["reasoning_tokens"],
                completion_tokens=meta["completion_tokens"])

        choices = data.get("choices") or []
        if choices and isinstance(choices[0], dict):
            choice = choices[0]
            message = choice.get("message") if isinstance(choice.get("message"), dict) else {}
            # A few compatible gateways put the delta/text directly on choice.
            text = _content_text(message.get("content"))
            if not text:
                text = _content_text(choice.get("text") or choice.get("delta"))
            finish = str(choice.get("finish_reason") or
                         choice.get("finishReason") or "")
            meta["finish_reason"] = finish
            meta["truncated"] = finish == "length"
            if text:
                return text, meta
            refusal = _content_text(message.get("refusal"))
            raise _completion_error(
                p, finish_reason=finish, refusal=refusal,
                reasoning=_reasoning_text(message),
```

5.2 LLM Router — rate-limit cooldown (app/api/llm_router.py)

```
# ---- app/api/llm_router.py (excerpt) ----
def _note_rate_limit(p: Provider, retry_after_s: float = 0.0) -> None:
    global _rate_limit_until
    if not _is_openrouter(p):
        return
    wait = retry_after_s if retry_after_s > 0 else settings.openrouter_429_cooldown_s
    with _rate_limit_lock:
        _rate_limit_until = max(_rate_limit_until, time.time() + max(0.0, wait))

def _wait_out_rate_limit(p: Provider) -> None:
    """Block until the shared OpenRouter cooldown has expired (bounded)."""
    if not _is_openrouter(p):
        return
    with _rate_limit_lock:
        remaining = _rate_limit_until - time.time()
    wait = min(max(0.0, remaining), _MAX_COOLDOWN_WAIT_S)
    if wait > 0:
        logger.info("OpenRouter rate-limit cooldown: waiting %.0fs before next call", wait)
        time.sleep(wait)

def _reset_rate_limit_state() -> None:
    """Test hook: clear the shared cooldown."""
    global _rate_limit_until
    with _rate_limit_lock:
        _rate_limit_until = 0.0

def resolve_slot(slot: str) -> str:
    """Translator inherits the Writer slot when it has no key of its own."""
    if slot == "translator":
        if get_credentials("translator"):
            return "translator"
        return "writer"
    return slot

def is_configured(slot: str) -> bool:
    return get_credentials(resolve_slot(slot)) is not None

def configured_slots() -> List[str]:
```

5.3 Agent runtime — truncated-JSON repair (app/agents/base.py)

```
# ---- app/agents/base.py (excerpt) ----
def extract_json(text: str) -> Optional[Any]:
    """Best-effort JSON extraction from an LLM response.

    Free models commonly wrap JSON in prose/fences or stop mid-object when
    their output budget runs out. A complete embedded object is decoded
    first; a truncated object is then closed (dropping only an incomplete
    trailing property/item if necessary). This never invents values — every
    retained token came from the provider — and agent-level key validation
    still decides whether the result is usable.
    """
    if not text:
        return None
    cleaned = text.strip().lstrip("\ufeff")
    if cleaned.startswith("```"):
        cleaned = re.sub(r"^```(?:json)?\s*", "", cleaned, flags=re.I)
        cleaned = re.sub(r"\s*```$", "", cleaned).strip()
    try:
        return json.loads(cleaned)
    except json.JSONDecodeError:
        pass

    decoder = json.JSONDecoder()
    starts = [m.start() for m in re.finditer(r"[\[{]", cleaned)]
    for start in starts[:12]:
        fragment = cleaned[start:]
        try:
            value, _ = decoder.raw_decode(fragment)
            return value
        except json.JSONDecodeError:
            pass
        candidate = _close_json_fragment(fragment)
        try:
            return json.loads(candidate)
```

```
        except json.JSONDecodeError:
            pass
        # If the response stopped in the middle of a property/item, retry
        # with progressively earlier complete comma boundaries.
        search = fragment
        for _ in range(20):
            comma = search.rfind(",")
            if comma <= 0:
                break
            search = search[:comma]
            try:
                return json.loads(_close_json_fragment(search))
            except json.JSONDecodeError:
                continue
    return None

@dataclass
class AgentResult:
    agent: str
    ok: bool
    data: Dict[str, Any]
```

5.4 Multi-agent orchestrator (app/agents/orchestrator.py)

```
# ---- app/agents/orchestrator.py (excerpt) ----
class Orchestrator:
    """Executes the agent graph and emits mission-control events."""

    def __init__(self, report_id: str = "", emit: Optional[Callable[[dict], None]] = None):
        self.report_id = report_id
        self.emit = emit or (lambda _e: None)
        self.events: List[dict] = []
        self.results: List[AgentResult] = []

    def _push(self, event: dict) -> None:
        event = dict(event)
        event.setdefault("ts", time.time())
        self.events.append(event)
        try:
            self.emit(event)
        except Exception:
            pass

    def _run_agent(self, name: str, state: Dict[str, Any]) -> AgentResult:
        meta = AGENT_META[name]
        self._push({"type": "agent_start", "agent": name, "name": meta["name"],
                    "icon": meta["icon"], "status": "working"})
        started = time.perf_counter()
        try:
            agent = AGENT_CLASSES[name](self.report_id, self._push)
            result = agent.run(state)
        except AnswerUnavailable as exc:
            # Verified mode must fail closed. Never replace a missing/failed
            # API or missing evidence with a plausible template result. Emit
            # a terminal event first: without it Mission Control left the
            # agent at WORKING forever after the job had already failed.
            elapsed_ms = int((time.perf_counter() - started) * 1000)
            self._push({"type": "agent_failed", "agent": name,
                        "name": meta["name"], "icon": meta["icon"],
                        "status": "failed", "elapsed_ms": elapsed_ms,
                        "error": exc.message, "reason": exc.reason,
                        "hint": exc.hint})
            try:
                queries.add_agent_log(self.report_id, name, "configured provider",
                                      "failed", elapsed_ms, 0, 0, exc.message)
            except Exception:
                pass  # telemetry must never mask the real failure
            raise
        except Exception as exc:
            logger.exception("Agent %s failed hard; using safe fallback", name)
            result = AgentResult(agent=name, ok=False, data={"fallback": True},
                                 warning=f"{name} recovered from a critical error: {exc}")
            result.data["_meta"] = {"agent": name, "ok": False, "source": "fallback",
                                    "warning": result.warning, "latency_ms": 0,
```

```
                                "provider": "recovery", "model": "n/a", "slot": name}
        self.results.append(result)
        self._push({"type": "agent_done", "agent": name, "name": meta["name"],
                    "status": "done" if result.ok else "recovered",
                    "elapsed_ms": int((time.perf_counter() - started) * 1000),
                    "provider": result.provider,
                    "warning": result.warning})
        return result

    def run(self, query: str, *, region: str = "Global", period: str = "3Y",
            language: str = "en") -> Dict[str, Any]:
        state: Dict[str, Any] = {
            "query": query, "region": region, "period": period,
            "language": language, "report_id": self.report_id,
        }
        total = 7
        done = 0
        self._push({"type": "start", "query": query, "region": region,
                    "period": period, "total_agents": total})

        def progress(stage: int) -> None:
            self._push({"type": "progress", "stage": stage, "completed": done,
                        "total": total, "percent": round(done / total * 100)})

        # Stage 1 — research
        res = self._run_agent("researcher", state)
        state["researcher"] = res.data
        state.setdefault("dataset", res.data.get("dataset", {}))
        done += 1
        progress(1)

        # Stage 2 — analysis
        res = self._run_agent("analyst", state)
        state["analyst"] = res.data
        done += 1
        progress(2)

        # Stage 3 — parallel fan-out
        parallel = ("writer", "forecast", "factchecker")
        with ThreadPoolExecutor(max_workers=3) as pool:
            futures = {pool.submit(self._run_agent, n, dict(state)): n for n in parallel}
            for fut in as_completed(futures):
                name = futures[fut]
                try:
                    state[name] = fut.result().data
                except AnswerUnavailable:
                    raise
                except Exception as exc:
                    state[name] = {"fallback": True, "error": str(exc)}
                done += 1
                progress(3)
        # Forecast needs its own output in ctx for LLM enrichment; give it a second,
        # cheap narrative pass only when a provider is configured (handled in merge).
        state["agent_meta"] = [r.as_dict() for r in self.results]

        # Stage 4 — chat agent readiness (no generation cost)
        self._push({"type": "agent_start", "agent": "chat", "name": "Chat Agent",
                    "icon": "■", "status": "working"})
        time.sleep(0.05)
        state["chat"] = {"ready": True, "suggestions": ChatAgent.SUGGESTIONS}
        self._push({"type": "agent_done", "agent": "chat", "name": "Chat Agent",
                    "status": "done", "elapsed_ms": 50,
                    "provider": "Meridian Local Engine", "warning": ""})
        done += 1
        progress(4)

        # Stage 5 — translation (conditional edge)
        self._push({"type": "agent_start", "agent": "translator", "name": "Translator",
                    "icon": "■", "status": "working"})
        try:
            if language != "en":
```

5.5 Analyst agent — grounded strict prompt (app/agents/analyst.py)

```
# ---- app/agents/analyst.py (excerpt) ----
```

```
    def strict_prompt(self, ctx: Dict[str, Any], scaffold: Dict[str, Any]) -> str:
        sources = (ctx.get("dataset", {}) or {}).get("verified_sources") or []
        return (
            "Rewrite the strategic analysis for a market-intelligence report. "
            "Be terse so the JSON finishes. Return JSON with keys: summary "
            "(max 45 words), swot (object with Strengths/Weaknesses/"
            "Opportunities/Threats arrays of exactly 2 strings, each max 10 "
            "words), pestle (6 objects factor/impact/note, note max 12 words), "
            "porter (5 objects force/intensity/note, note max 12 words), "
            "trends (max 3 objects trend/direction/confidence), key_metrics "
            "(max 4 objects label/value/delta/tone — use 'not available' for "
            "values), strategic_priorities (exactly 3 strings, max 8 words "
            "each), growth_drivers (exactly 3 strings, max 8 words each).\n\n"
            "CRITICAL: no number in the context is measured. Use ONLY the VERIFIED "
            "SOURCES; if they do not support a claim, say 'not available'. Never "
            "invent a figure, publisher, URL or citation.\n\n"
            f"QUERY: {compact(ctx.get('query', ''), 300)}\n"
            f"VERIFIED SOURCES:\n{compact(sources, 3200)}\n"
        )

    def llm_prompt(self, ctx: Dict[str, Any]) -> str:
        d = ctx.get("dataset", {})
        return (
            "As the Analyst agent, sharpen the strategic interpretation of this dataset.\n"
            "Return JSON with keys: summary (2 sentences), additional_trends (array of 2 objects "
            "with trend/direction/confidence), strategic_priorities (array of 3 short strings). "
            "Stay strictly grounded in the supplied numbers.\n\nDATA:\n" + compact({
                "company": d.get("company"), "sector": d.get("sector"),
                "region": d.get("region"), "market_size_usd_b": d.get("market_size_usd_b"),
                "cagr_pct": d.get("cagr_pct"), "competitors": d.get("competitors"),
```

5.6 Encrypted key vault (app/api/key_manager.py)

```
# ---- app/api/key_manager.py (excerpt) ----
def save_key(agent_name: str, provider: str, api_key: str, model: str = "",
             is_active: bool = True) -> Dict[str, Any]:
    """Validate, encrypt and persist a key for one of the six agent slots."""
    from app.config import AGENT_SLOTS
    if agent_name not in {slot[0] for slot in AGENT_SLOTS}:
        raise ValueError(f"Unknown agent slot: {agent_name}")
    p = get_provider(provider)
    clean = (api_key or "").strip()
    if clean:
        _validate_key_shape(p.label, clean)
    if not clean and provider != "ollama":
        # The Settings form promises "leave blank to keep current". Preserve
        # the already-encrypted credential for this same slot instead of
        # rejecting a provider/model-only change.
        existing = queries.get_api_key_row(agent_name)
        if existing:
            clean = decrypt(existing["encrypted"])
            if clean == "local-no-key-required":
                clean = ""
        if not clean:
            raise ValueError("API key cannot be empty")
    effective_model = normalize_model(provider, model or p.default_model)
    # OpenRouter's live free catalogue is accepted too (any :free slug), while
    # a paid model can never be pinned on a free provider by mistake. Fail
    # loudly instead of silently saving a different model than the user chose.
    if not is_allowed_model(provider, effective_model):
        if provider == "openrouter_free":
            raise ValueError(
                "That model is not a free OpenRouter variant. Choose "
                "“Auto — best free model (live)”, “OpenRouter free router”, "
                "or a model whose ID ends in :free.")
        raise ValueError(
            f"Model '{effective_model}' is not available for {p.label}. "
            "Choose one of the listed models.")
    token = encrypt(clean) if clean else encrypt("local-no-key-required")
    return queries.upsert_api_key(agent_name, provider, token, effective_model, is_active)

def get_credentials(agent_name: str) -> Optional[Dict[str, Any]]:
    """Return decrypted config for internal use only."""
    row = queries.get_api_key_row(agent_name)
```

```
    if row is None or not row["is_active"]:
        return None
    try:
        secret = decrypt(row["encrypted"])
```

5.7 Report job service (app/services.py)

```
# ---- app/services.py (excerpt) ----
def _worker(report_id: str, query: str, region: str, period: str, language: str) -> None:
    job = _JOBS[report_id]
    def emit(event: dict) -> None:
        with job.cond:
            job.events.append(event)
            job.cond.notify_all()
    try:
        orch = Orchestrator(report_id, emit)
        state = orch.run(query, region=region, period=period, language=language)
        _persist_state(report_id, state)
        emit({"type": "exporting", "message": "Building PDF, PowerPoint and Excel exports"})
        try:
            generate_exports(report_id)
        except Exception as exc:
            logger.exception("Export stage failed")
            emit({"type": "warning", "message": f"Exports unavailable: {exc}"})
        job.status = "completed"
        job.finished_at = time.time()
        emit({"type": "job_complete", "report_id": report_id, "status": "completed"})
    except Exception as exc:
        logger.exception("Report job %s failed", report_id)
        job.status = "failed"
        job.error = str(exc)
        job.finished_at = time.time()
        try:
            queries.update_report(report_id, status="failed", error_message=str(exc))
        except Exception:
            pass
        payload = {"type": "job_failed", "report_id": report_id, "error": str(exc),
                   "trace": traceback.format_exc()[-1200:]}
        if isinstance(exc, answer_policy.AnswerUnavailable):
            payload.update({"error_type": "answer_unavailable", "reason": exc.reason,
                            "hint": exc.hint,
                            "setup_path": exc.as_dict()["setup_path"]})
        emit(payload)

def start_report(query: str, *, region: str = "Global", period: str = "3Y",
                 language: str = "en", company: str = "") -> str:
    """Create a report and run the agent graph in a background thread."""
```

5.8 Frontend mission control (app/web/app.js)

```
// ---- app/web/app.js (excerpt) ----
function connectEvents(id) {
  if (S.eventSource) { S.eventSource.close(); S.eventSource = null; }
  try {
    S.eventSource = new EventSource(`/api/reports/${id}/events`);
    S.eventSource.onmessage = (ev) => {
      let e; try { e = JSON.parse(ev.data); } catch { return; }
      if (e.type === 'agent_start') { S.agents[e.agent] = {status:'working'}; S.jobStatus = 'running'; }
      else if (e.type === 'agent_done') S.agents[e.agent] = {status:e.status || 'done', provider:e.provider,
elapsed_ms:e.elapsed_ms};
      else if (e.type === 'agent_failed') S.agents[e.agent] = {status:'failed', elapsed_ms:e.elapsed_ms, error:e.
error || '', reason:e.reason || ''};
      else if (e.type === 'agent_retry') { /* quiet: Mission Control shows the retry state; a toast per retry stacked
over the failure toast */ S.agents[e.agent] = {...(S.agents[e.agent] || {}), status:'working', retry_note:'Retrying
after a provider hiccup…'}; }
      else if (e.type === 'progress') S.progress = e.percent;
      else if (e.type === 'warning' || e.type === 'agent_warning') toast(e.message || 'An agent recovered from a
provider issue', 'info');
      else if (e.type === 'job_complete') {
        S.jobStatus = 'done'; S.progress = 100; renderMission();
        toast('Report complete — PDF, PowerPoint and Excel are ready', 'ok');
        S.eventSource?.close(); S.eventSource = null;
        if (S.view === 'dashboard' || S.view === 'history') navigate('report', id);
        else if (S.view === 'report' && S.reportId === id) loadReport(id);
```

```
        refreshChrome(); return;
      } else if (e.type === 'job_failed') {
        S.jobStatus = 'failed'; renderMission();
        toast(`Report failed: ${e.error}${e.hint ? ` ${e.hint}` : ''}`, 'err', 8000);
        S.eventSource?.close(); return;
      }
      renderMission();
    };
    S.eventSource.onerror = () => { S.eventSource?.close(); S.eventSource = null; };
  } catch (e) { /* SSE unsupported: status polling still works */ }
}

/* ---------------------------------------------------------------- Views --- */
async function renderDashboard() {
  const reports = S.reports.slice(0, 5);
  const stats = S.stats || {};
  const ds = S.dashboard?.dataset;
  const suggestions = ['Indian electric vehicle market 2026', 'Global AI platforms competitive landscape',
    'Fintech payments in India', 'Semiconductor supply chain outlook', 'Renewable energy in Europe'];
  const gate = answerGateHtml();
  const canAnswer = answersEnabled();
  let html = `${gate}
  <div class="hero">
    <div class="hero-eyebrow">Multi-agent market intelligence</div>
    <h1>One query.<br><span>The entire market landscape.</span></h1>
    <p class="hero-sub">Six autonomous AI agents and a translator work in concert to research, analyse,
       write, forecast and fact-check a board-ready report — with live charts and instant exports.</p>
```

CHAPTER 6: SCREENSHOTS OF PROJECT

The following screenshots were captured directly from the live running Meridian web application during real report-generation sessions:

6.1 Dashboard — Query Composer & KPI Cards

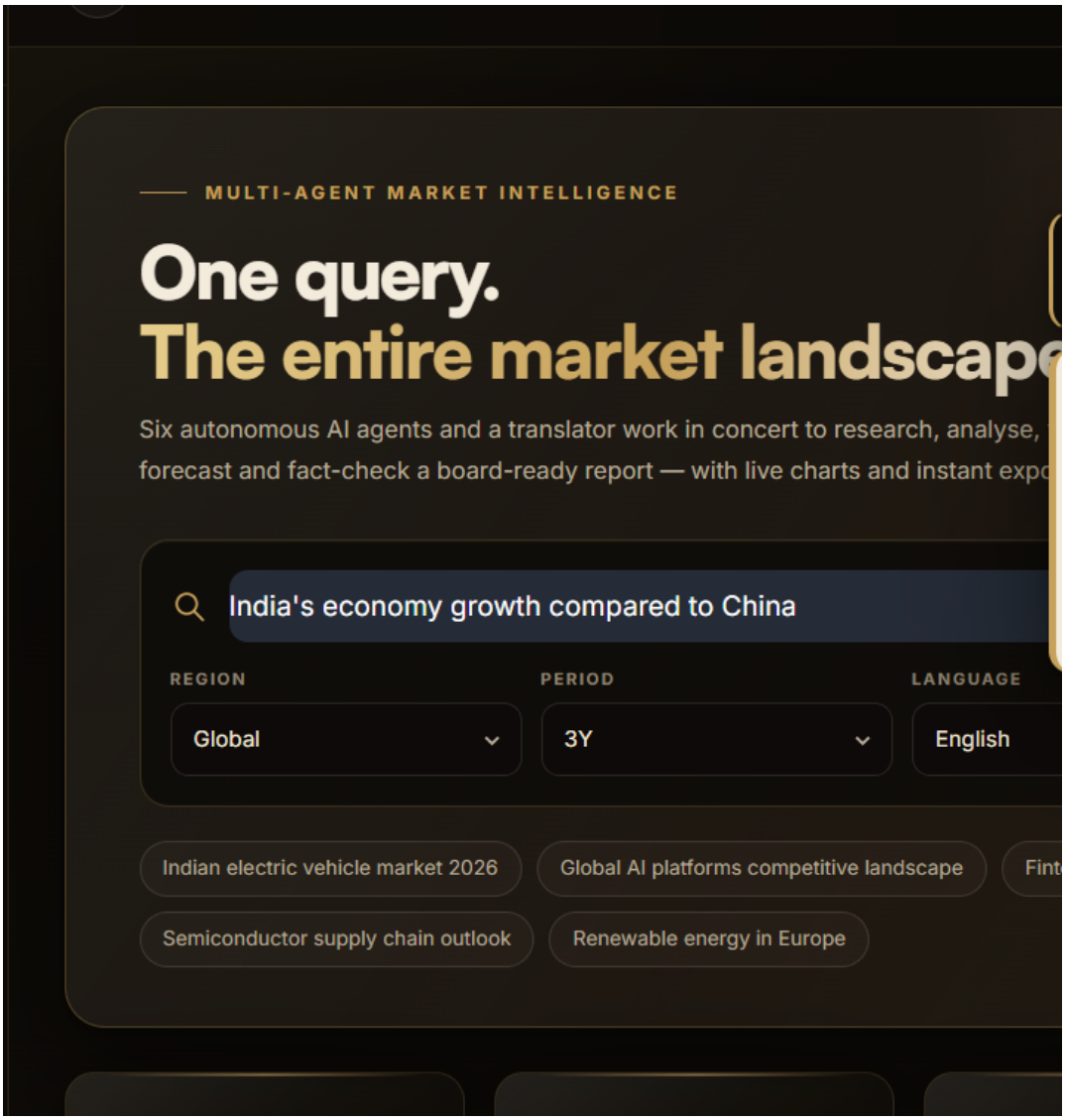


Figure 6.1: Meridian Dashboard showcasing the one-query composer with Region / Period / Language selectors, suggestion chips, and KPI cards in the Obsidian + Champagne + Pearl aesthetic.

6.2 Live Generation — Agent Activity (Mission Control)

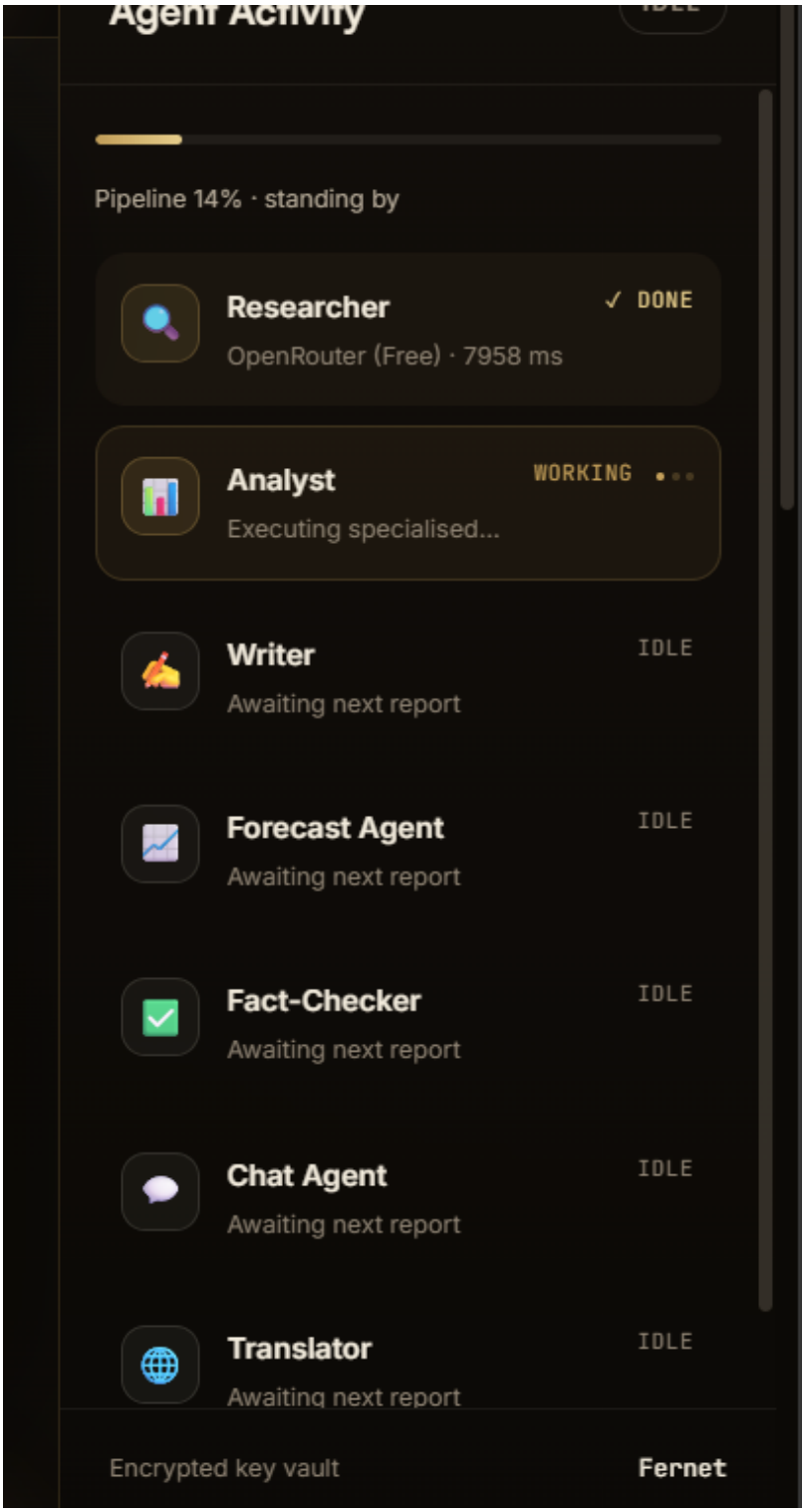


Figure 6.2: Agent Activity panel streaming per-agent status — Researcher, Analyst, Writer, Forecast Agent, Fact-Checker, Chat Agent, Translator — with pipeline progress and terminal FAILED/DONE states.

6.3 Report Workspace



Figure 6.3: The report workspace during generation, showing the submitted query and live pipeline state before the cited report opens.

6.4 API Keys — Independent Agent Slots (Settings)

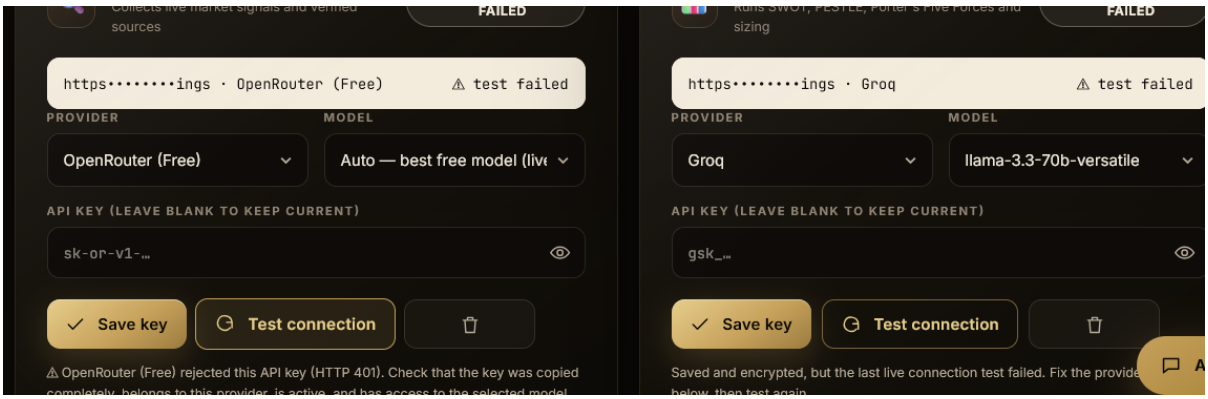


Figure 6.4: Settings → API Keys displaying independent encrypted slots per agent with provider/model selection and Test connection, which probes the typed key and saves it only when it works.

6.5 Generation Telemetry & Counters

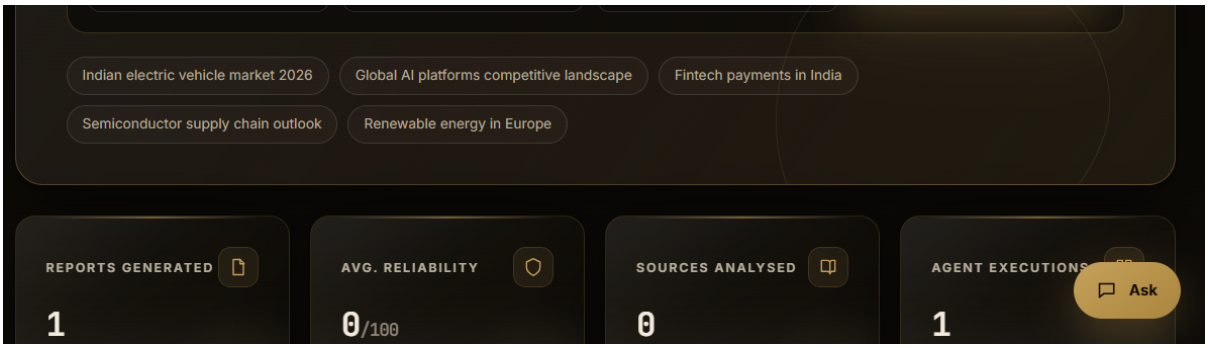


Figure 6.5: Live counters — Reports completed, Avg. reliability, Sources analysed (counted from actually stored sources), and Agent executions.

CHAPTER 7: SOFTWARE TESTING & ERROR HANDLING

7.1 Testing Strategy

Meridian is verified by an automated pytest suite of 123 tests covering the provider catalogue and router, the agent pipeline, the verified-answer policy, the encrypted key vault, database statistics, exports, deployment configuration, and responsive UI contracts. All provider HTTP traffic is mocked, so the suite runs offline with no API key. Every production failure discovered during development was converted into a permanent regression test before the fix shipped.

7.2 Test Cases

Test ID	Module	Test Case	Result
TC-01	OpenRouter catalogue	Only explicit :free models with all-zero pricing are auto-selected; paid and funded-account $0 models are rejec	Pass
TC-02	LLM router	A retired free model (HTTP 404) fails over to the next free model; a paid model is never sent.	Pass
TC-03	LLM router	Reasoning-only completion (finish_reason=length, empty content) is typed as truncated and retried with reason	Pass
TC-04	LLM router	HTTP 429 opens a shared cooldown; the next call waits out Retry-After instead of re-tripping the limit.	Pass
TC-05	Agent runtime	Truncated JSON is repaired; malformed output triggers up to 3 strict-JSON retries avoiding the bad model.	Pass
TC-06	Agent runtime	A rate-limited agent falls back to another configured free provider; a paid provider is never substituted.	Pass
TC-07	Answer policy	With no API key, report/chat/translation/compare are refused with HTTP 409 answer_unavailable.	Pass
TC-08	Key vault	Blank save preserves the stored key; a URL pasted as a 'key' is rejected with actionable guidance.	Pass
TC-09	Key vault	Test connection probes the typed key and saves it only on success; a failing key never overwrites the stored on	Pass
TC-10	Orchestrator	A failed agent emits a terminal agent_failed event; Mission Control shows FAILED instead of stuck WORKIN	Pass
TC-11	Dashboard stats	Failed reports do not inflate source counters; sources are counted from stored verified sources.	Pass
TC-12	Exports	PDF, PowerPoint, and Excel exports generate for a completed report.	Pass
TC-13	Responsive UI	No bottom tab bar; settings show 7 slots with honest saved/tested state badges.	Pass

7.3 Error Handling

- **HTTP 401/403 (invalid key):** Fail fast with copy-complete-key guidance; auth failures are never retried across models and never leak the secret.
- **HTTP 402 (OpenRouter billing gate):** Explained as an account/per-key credit-limit issue affecting even :free routing — not a broken key field — with concrete remedies.
- **HTTP 429 (rate limit):** Free-model failover within the call, a shared cooldown honouring Retry-After between calls, serialised OpenRouter requests, and an optional rescue by another configured free provider.

• **Empty / truncated / reasoning-only completions:** Typed completion errors, reasoning-disabled retry, JSON repair, and shorter strict-JSON agent retries — then an honest failure, never a template answer.

• **Missing evidence:** Reports are refused (409) rather than published with invented figures; unverified chart series are labelled as placeholders.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Conclusion

Meridian proves that a student-built SaaS product can deliver consulting-grade market intelligence at zero recurring cost by combining foundational web technologies with a disciplined multi-agent AI design. Its defining contribution is trust: a verified-answer policy that refuses to fabricate, claim-level fact-checking with reliability scores, genuine free-model auto-selection, encrypted credential management, and production-hardened handling of the empty-completion, truncation, billing-gate, and rate-limit failures that free-tier AI exhibits in the real world. The complete system — seven agents, sixteen providers, fourteen languages, three export formats — is covered by 123 automated tests and deployed as a responsive web application.

8.2 Future Scope

- **Scheduled & Comparative Intelligence:** Periodic re-generation of saved reports with change detection and trend alerts delivered by email.
- **Deeper Evidence Graph:** Claim-to-source linking at sentence level with an evidence browser showing the exact passage behind every figure.
- **Team Workspaces:** Shared report libraries, comments, and role-based access for organisations, with per-team key vaults.
- **Streaming Writer:** Token-streamed report drafting with section-level regeneration and tone controls.
- **Offline Model Option:** Optional local inference (Ollama) for fully private deployments where no external API may be used.

REFERENCES

• MDN Web Docs — HTML5, CSS3 & Modern JavaScript Reference: https://developer.mozilla.org/

• Python 3 Official Documentation — Python Software Foundation: https://docs.python.org/3/

• FastAPI Official Documentation — Sebastián Ramírez: https://fastapi.tiangolo.com/

• OpenRouter API & Free Models Documentation: https://openrouter.ai/docs

• Groq Cloud OpenAI-Compatible API Reference: https://console.groq.com/docs

• Google AI Studio — Gemini API Official Documentation: https://ai.google.dev/gemini-api/docs

• W3Schools Online Web Tutorials (HTML, CSS, JS & Python): https://www.w3schools.com/

• ReportLab PDF Library Documentation: https://docs.reportlab.com/

• Streamlit Official Documentation: https://docs.streamlit.io/

• python-pptx Documentation: https://python-pptx.readthedocs.io/

• openpyxl Documentation: https://openpyxl.readthedocs.io/

• Render Cloud Hosting Documentation: https://render.com/docs

• IK Gujral Punjab Technical University: https://ptu.ac.in/